

App Dev 2 Project Report

1. Student Details

- **Name:** Deva Vasista
- **Roll Number:** 21f2000226
- **Email:** 21f2000226@ds.study.iitm.ac.in
- **About Me:** I am a working professional and an AI/ML Engineer currently pursuing the Diploma level in the IIT Madras Online Degree Program. I enjoy building practical applications and designing systems end-to-end, from databases and backend logic to user interfaces. I am also interested in working with cloud technologies and applying AI/ML to solve real-world problems.

2. Project Details

Project Title: Katalyst – Placement Portal Application

Problem Statement:

Institutes often manage campus recruitment with spreadsheets and ad-hoc email, which makes company approvals, student applications, and placement records hard to track. This project builds a role-based Placement Portal for Admin (Institute), Company, and Student users so the full hiring flow can run in one system.

Approach:

Katalyst is a decoupled client-server application with a Flask JSON API backend and a Vue 3 frontend. The backend uses SQLite, JWT authentication, Redis caching, and Celery for background tasks, while the frontend communicates through REST API endpoints. The entire stack runs locally with Vite, Bootstrap, and MailHog for development and testing.

The system has three distinct user groups with separate dashboards. Admins approve companies and drives, manage records, and trigger automated tasks. Companies create drives, review applicants, conduct interviews, and select candidates, while students browse approved drives, apply if eligible by CGPA, and manage their profile and resume.

3. AI/LLM Declaration

Tool used: Claude Haiku 4.5

Area	Purpose	AI %	Notes
Backend (Flask + API)	API route design, blueprint structure, JWT/auth patterns, Celery task setup	20%	Flask patterns, JWT middleware setup, Celery task structure suggestions
Database Schema	SQLAlchemy model design, column naming, relationship structure	12%	Model boilerplate, column definitions, table relationships, ORM configuration
Frontend UI (Vue + Bootstrap)	Component structure, state management, responsive layout, styling	15%	Vue component templates, Bootstrap grid layouts, routing setup, form components
Testing & Deployment	Local environment setup, configuration files	5%	Docker/environment config guidance, Redis and MailHog setup instructions
Total	-	35 – 40%	-

All core business logic, security implementations, integration work, bug fixes, debugging, and final design decisions were completed independently.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Python	Core language for the Backend Flask API and Celery tasks.
Flask	Main web framework for backend REST API, Routing, blueprints
Flask-JWT-Extended	Gather Access/refresh tokens and role claims
Flask-Bcrypt	Secure password hashing for user credentials.
SQLAlchemy	ORM used to define models and interact with the SQLite database.
SQLite	Lightweight relational database for development and testing.
Redis + Flask-caching	Caching frequently accessed data
Celery	Running background tasks (emails, reports, exports)
Vue 3	Main frontend user interface
Vue Router	Navigation and route protection
Vite	Development environment for the frontend
Bootstrap 5	Responsive design and styling
MailHog	Testing email functionality locally

This technology stack was chosen because it's practical for a coursework project, easy to run locally, and clearly separates the API from the user interface.

5. Database Schema / ER Design

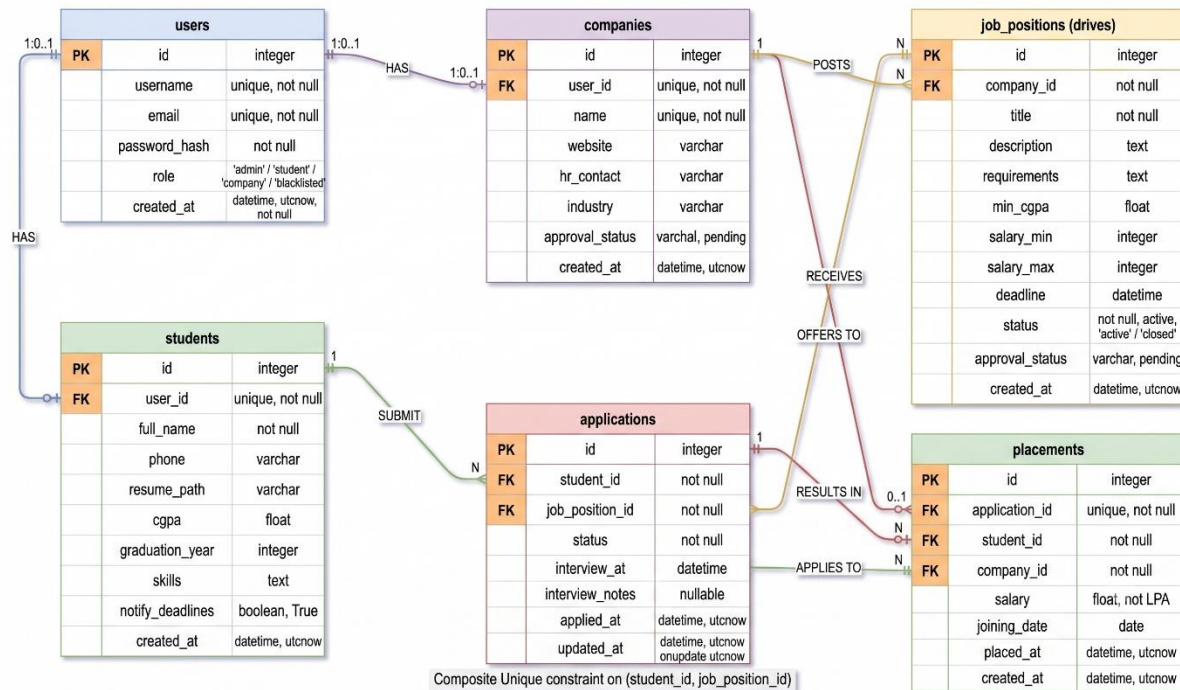
The database has six tables: User, Student, Company, Drive, Application, and Placement.

Tables

Table	Key columns / purpose
User	id, username, email, password_hash, role, created_at. Stores login and role for every account (admin, student, company).
Student	id, user_id (FK), full_name, phone, resume_path, cgpa, graduation_year, skills, notify_deadlines. Student profile linked to one User.
Company	id, user_id (FK), name, website, hr_contact, industry, approval_status. Company profile linked to one User; must be approved before posting drives.
Job Position	id, company_id (FK), title, description, requirements, min_cgpa, salary_min, salary_max, deadline, status, approval_status. Job posting created by a company; requires admin approval.
Application	id, student_id (FK), job_position_id (FK), status, interview_at, interview_notes, applied_at. Links a student to a drive; one application per (student, drive) pair.

Table	Key columns / purpose
Placement	id, application_id (FK), student_id (FK), company_id (FK), salary, joining_date. Final placement record created when a company selects a student.

ER Diagram:



Design Rationale:

The Database Schema was designed to separate authentication (User table) from role-specific profiles (Student, Company), so login logic remains simple and secure. I structured intermediate tables (Job Position, Application, Placement) to mirror the real recruitment pipeline, making it intuitive to track the flow from posting to selection. I added explicit status and approval fields on drives and applications so admins and companies can control visibility and manage progress without duplicating data. This approach keeps the database clean, scalable, and aligned with how campus placements actually work.

6. API Design (Routes)

The frontend communicates with the backend exclusively through REST API endpoints. All protected routes require JWT tokens in the Authorization header. The complete request and response specifications are documented in `api.yaml` using OpenAPI 3.0 format.

Summary:

- **Health Check:** Verify that the API server and Redis cache are running and responding correctly.
- **Authentication:** Handle user registration and login, issue and refresh JWT access tokens, and retrieve the currently logged-in user's profile and role information.

- **Public Endpoints:** Allow unauthenticated users to browse all approved and active job drives, view company details and profiles, and search across available positions.
- **Admin Functions:** Display dashboard statistics and analytics, approve or reject company registrations and job postings, blacklist or manage user accounts, create or delete student and company records, and retrieve all confirmed placements across the system.
- **Company Functions:** Create, edit, and manage job drives, review incoming student applications with filtering options, update application status (shortlist, select, or reject), view student resumes, and edit company profile information.
- **Student Functions:** Access a personalized dashboard with application status, manage their profile and skills, apply to or withdraw from job drives, check CGPA eligibility against drive requirements, and upload or delete their resume.
- **Background Jobs:** Queue asynchronous Celery tasks for sending daily deadline reminders, generating monthly placement reports, and exporting application data as CSV files. Poll task status to check completion or download the generated file directly.

7. Architecture and Features

Project Structure

- **backend/:** Flask JSON API with SQLite, JWT auth, Redis caching, and Celery jobs.
 - app.py: Entry point that starts the server and can seed the database if it is missing.
 - __init__.py, config.py, and extensions.py: App factory, settings, and shared objects for the database, JWT, and cache.
 - models.py: Defines User, Student, Company, JobPosition, Application, and Placement.
 - auth/: Login helpers, JWT claims, and role guards.
 - routes/: API blueprints for auth, admin, company, student, public, jobs, and health.
 - utils/: JSON serializers and request parsers.
 - cache.py: Helpers for Redis caching.
 - tasks.py and workers.py: Background jobs for reminders, interview mail, the monthly report, and CSV export.
 - seed_db.py: Loads demo data for local demos.
 - static/resumes/: Stores uploaded student resumes.
 - api.yaml: OpenAPI 3.0 description of the API.
- **frontend/:** Vue 3 Composition API SPA built with Vite and Bootstrap.
 - index.html and vite.config.js: SPA shell and proxy that forwards /api requests to Flask.
 - src/main.js and App.vue: Boot the Vue app, navbar, router outlet, and toasts.
 - src/api/: HTTP client that attaches the Bearer token.
 - src/services/: Auth state for login, me, and logout, plus toast messages.
 - src/router/: Page routes and role-based guards.
 - src/components/: Shared UI such as the navbar, sidebar layout, pager, and status badges.
 - src/views/: Pages for home, login, and register, plus admin, company, and student dashboards.

Key Features Implemented

- **Decoupled Architecture:** The frontend is a Vue single-page application that communicates with the backend entirely through REST APIs. The Flask backend returns only JSON data, never HTML, which keeps the two layers independent and makes the system easier to maintain and scale.
- **Role-Based Access Control:** I implemented JWT token-based authentication with role guards on both the API and frontend routes. This ensures that students can't access admin features, companies can't modify student records, and blacklisted users are blocked immediately from the system.
- **Admin Dashboard:** Admins can approve or reject companies before they're allowed to post drives, and they can approve or reject job postings from companies. They have full visibility into all students, companies, and drives in the system. Admins can also view all confirmed placements and trigger automated tasks like daily reminder emails and monthly placement reports through background jobs.
- **Company Features:** Companies can create and manage job postings (pending admin approval) and set minimum CGPA requirements to filter eligible applicants. They can review student applications with full access to resumes, shortlist candidates with interview scheduling capabilities, and finally select candidates by entering salary and joining date details. Companies can also export application data as CSV for record-keeping.
- **Student Experience:** Students can create and maintain their profile with skills and contact information, browse all active and approved job postings, and apply only if they meet the CGPA eligibility requirement. They can track their applications, see interview schedules, upload and manage their resume, and optionally receive email reminders about upcoming application deadlines.
- **Performance and Background Tasks:** I implemented Redis caching for frequently accessed data like job listings to reduce database load and improve response time. Celery handles asynchronous tasks in the background, including sending daily reminder emails to students, interview confirmation emails to selected candidates, generating monthly placement reports, and exporting application data as CSV files. *MailHog* captures all emails in local demonstrations without requiring a real mail server

8. Video Presentation

Drive Link: <https://drive.google.com/file/d/1jUYWiInIb65-8CgQXaC5k652OLc6XXWb/view?usp=sharing>